INTERVIEWHUB

Full Project Generation Prompt

A comprehensive specification for a complete full-stack interview platform

PRODUCT	Cal.com-style scheduling + live interviews + collaborative coding
FRONTEND	React · JavaScript · Vite · Tailwind CSS
BACKEND	Node.js · Express · MongoDB · Socket.IO
AUTHENTICATION	Google OAuth · email OTP · secure sessions
AUTOMATION	Email notifications · durable reminders · background workers
QUALITY	Security · testing · deployment · documentation · no fake functionality

How to use this document: Copy the project specification sections into your coding agent or AI development tool. Ask it to inspect the existing repository, implement the application incrementally, run tests, and report honestly on integrations that require credentials.

Core principle: No hardcoded runtime user data, fake analytics, fake meeting links, committed secrets, or false claims that integrations work.

Contents

1. Role and objective
2. Product vision and user roles
3. Technology stack
4. Non-negotiable: no hardcoded runtime data
5. Frontend pages and experience
6. Authentication and account lifecycle
7. Authorization and application security
8. Scheduling and availability engine
9. Email, calendar invitations, OTP and reminders
10. Meetings, real-time communication and WebRTC
11. Collaborative coding and safe execution
12. Question bank, feedback and assessments
13. Analytics and reporting
14. MongoDB data model
15. API design and error contract
16. Testing requirements
17. Observability, reliability and deployment
18. Suggested repository structure
19. Delivery requirements and definition of done
20. Execution instructions for the coding agent

The following sections form the master prompt. Give the complete document to the coding agent, not just the table of contents.

1. Role and objective

You are a senior full-stack software architect, product designer, security engineer, and implementation engineer. Build a complete, polished, production-style web application named **InterviewHub**: a technical interviewing platform combining Cal.com-style scheduling, Google Meet-style live interviews, collaborative coding, structured feedback, and analytics.

Deliver a working application, not a mockup, static prototype, or collection of disconnected screens. Implement the frontend, backend, database models, integrations, validation, authorization, background jobs, tests, deployment configuration, seed/demo tooling, and detailed documentation. Every user-visible action must be functional or explicitly marked as unavailable; never fake success.

2. Product vision and user roles

The platform supports three roles: **Candidate**, **Interviewer**, and **Administrator**. A user may have more than one authorized role if the permission model supports it. Enforce roles and resource ownership on the backend, not only in the frontend.

Candidate capabilities: create and manage profile; browse public interviewer booking pages; book, reschedule, or cancel interviews; receive email confirmations and reminders; join permitted interview rooms; use coding assessments; view their own submissions and any feedback explicitly made visible to them.

Interviewer capabilities: configure profile, timezone, weekly availability, time off, event types, booking questions, buffers, minimum notice and booking horizon; publish and share booking links; manually schedule interviews; manage bookings; join interview rooms; create question sets; review submissions; complete private scorecards and feedback; view permitted analytics.

Administrator capabilities: manage accounts and role assignments; review system health, audit events, abuse reports, and operational metrics; manage global settings and templates; inspect failed notification jobs. Administrators must not automatically gain unrestricted access to private interview content without a documented permission and audit policy.

3. Technology stack

Use JavaScript throughout the application, with ES modules and a consistent module strategy. Frontend: React, Vite, React Router, Tailwind CSS, TanStack Query, React Hook Form, Zod, Monaco Editor, Recharts, and FullCalendar where appropriate. Use accessible reusable components and a coherent design system.

Backend: supported current Node.js LTS, Express, REST APIs, Socket.IO, Mongoose, Zod validation, structured logging, centralized error handling, and a modular service/controller/repository architecture where useful.

Data and jobs: MongoDB for persistent application data; Redis and BullMQ for durable background jobs, reminders, retries, and notification processing. Use transactions where supported and appropriate; otherwise implement explicit concurrency-safe patterns and document their guarantees.

Authentication and security: Google OAuth 2.0 / OpenID Connect, secure server-managed sessions using HTTP-only cookies, Argon2id password hashing, email OTP verification, CSRF protection where applicable, Helmet, rate limiting, input validation, and secure cookie settings.

Communications: choose one email provider such as Resend or SMTP via Nodemailer and implement it behind a provider adapter. Use reusable responsive email templates. Video/audio: WebRTC with STUN/TURN and Socket.IO for signaling. Code editor: Monaco Editor. Code execution: Judge0 or a comparable isolated execution service; never execute untrusted submissions in the main application server.

Testing and delivery: Vitest, Supertest, and Playwright; GitHub Actions CI; Docker where useful; frontend hosting such as Vercel, a Node-compatible backend host, and managed MongoDB/Redis. Choose compatible current stable package versions, pin them with a lockfile, and document any version-sensitive decisions.

4. Non-negotiable: no hardcoded runtime data

Do not hardcode users, candidate names, interviewer names, email addresses, booking dates, time slots, meeting URLs, analytics values, OTPs, credentials, API keys, environment-specific URLs, or success responses. All runtime business data must come from validated user input, persisted records, trusted identity claims, or real provider responses.

Do not create fake analytics or pretend that integrations succeeded. Seed data is allowed only through an explicit development/demo seed command and must be clearly identified as demo data. Production analytics must query real persisted records.

Store environment-specific settings in validated environment variables or a secret manager. Commit only an .env.example containing variable names and safe descriptions, never real secrets. Validate required configuration on startup and fail fast with actionable errors.

Avoid scattered magic numbers. Put configurable policies such as reminder offsets, OTP lifetime, booking horizon, buffers, session lifetime, upload limits, and rate limits in validated configuration or persisted settings according to their scope. Security-critical limits must retain safe server-enforced bounds; an administrator must not be able to disable essential protections accidentally.

Static UI labels, type definitions, algorithmic constants, and safe defaults may exist in source code. The requirement is to avoid hardcoded runtime business data and make appropriate policies configurable, persisted, and testable.

5. Frontend pages and experience

Build a responsive, polished SaaS interface with a professional dark navy/charcoal theme, subtle borders, consistent spacing, readable typography, blue primary actions, keyboard accessibility, visible focus states, and mobile/tablet/desktop layouts. Support loading, empty, error, success, disabled, and permission-denied states on every data-driven page.

Required public pages: landing page; login; registration; OTP verification; forgot/reset password; public interviewer/event booking page; booking confirmation; cancellation/rescheduling pages; privacy policy and terms placeholders that are clearly marked for review rather than represented as legal advice.

Authenticated pages: onboarding; role-aware dashboard; profile/settings; availability editor; time-off manager; event-type list/create/edit; booking calendar and list; booking details; question bank; assessment editor; live interview room; submission history; feedback/scorecards; analytics; notifications; admin console; audit log and failed-job view for authorized administrators.

Dashboard data must be fetched from APIs. Candidate dashboards show their upcoming/past interviews, permitted feedback, and own submissions. Interviewer dashboards show upcoming interviews, pending scorecards, event types, booking activity, and metrics derived from actual records. Admin metrics must be scoped, privacy-conscious, and computed from real data.

Availability editor: weekly recurring hours by weekday, timezone selection, date-specific overrides, time off, event duration, buffer before/after, minimum scheduling notice, maximum booking horizon, slot interval, and optional daily limits. Show validation and preview the resulting public booking page.

Public booking flow: load the event type by public slug; display interviewer/event details, duration and timezone; fetch date-specific available slots; collect configured booking questions; validate the candidate's email; confirm the slot; show a confirmation page and send the actual email. Public visitors should not need an account merely to book unless the event owner explicitly configures that requirement.

Live interview room: participant tiles, mic/camera controls, device selection, screen share, participant list, connection state, chat, elapsed timer, shared coding workspace, question panel, run-code output, and leave/end meeting actions. Only authorized participants may join. Make reconnecting and permission failures understandable to users.

Use React Hook Form and Zod (or shared schemas where feasible) for forms. TanStack Query should manage server state, caching, invalidation, retries and loading states. Never trust client-side validation as a security boundary.

6. Authentication and account lifecycle

Implement email/password registration and login plus Google OAuth using a secure authorization-code flow with state validation and PKCE where supported/appropriate. Validate OAuth callback URLs and provider identity claims. Never trust an email supplied by the browser as proof of Google identity.

For email/password registration, validate input, hash the password with Argon2id, create a pending or unverified account, generate a cryptographically secure OTP, store only a suitable hash and metadata, and send the OTP by email. OTP records must include purpose, user/email binding, expiry, attempt count, issue time, consumed status, and resend controls.

OTP rules: configurable short expiry (safe default around 5–10 minutes), single use, strict attempt limits, resend cooldown, rate limits by account and IP, invalidation of older codes when a new one is issued, constant-time comparison where

appropriate, and no OTP values in logs or API responses. Return generic responses where needed to reduce account enumeration.

After successful verification, mark the email verified and create a server-managed session. Use a cryptographically random session identifier in an HTTP-only, Secure-in-production, SameSite cookie. Store/revoke sessions server-side; rotate the session after login and privilege changes; expire idle and absolute sessions; implement logout and revoke-all-sessions where appropriate.

Implement forgot-password/reset flow with single-use, short-lived tokens or OTPs, rate limits, generic responses, and session invalidation after password changes. Implement account linking safely: never link identities solely because an unverified email string matches. Define behavior for existing accounts, disabled accounts, duplicate identities, and OAuth provider failures.

Google OAuth does not automatically require a second email OTP for every login. Require extra verification only where the configured security policy demands it, such as email/password registration or sensitive actions. Document the selected policy.

7. Authorization and application security

Use deny-by-default authorization. Every protected API must validate the authenticated session, role, resource ownership, and permitted state transition. Prevent IDOR/BOLA vulnerabilities: knowing a booking ID must not grant access to that booking.

Validate all inputs on the server, including path parameters, query parameters, body payloads, uploaded files and Socket.IO messages. Use safe database query construction; prevent NoSQL injection, mass assignment, XSS, CSRF, open redirects, SSRF, and unsafe file handling. Escape user-generated content in UI and email templates.

Configure CORS for explicitly permitted origins, secure cookie flags, HTTPS in production, security headers, body-size limits, upload limits, rate limiting, and safe error responses. Never expose stack traces, secrets, internal provider responses, or sensitive interview notes to clients.

Use least-privilege service credentials and restrict production database/network access. Redact passwords, OTPs, session tokens, OAuth codes, provider secrets, and sensitive personal information from logs. Keep audit events for security-sensitive actions without logging secrets.

Define retention and deletion policies for accounts, interview artifacts, code submissions, recordings (disabled unless explicitly implemented with consent), and audit logs. Do not record video/audio by default. Add consent and access controls before introducing recording.

8. Scheduling and availability engine

Event types are configurable records owned by an interviewer or organization. Fields may include title, description, slug, duration, location/meeting mode, active state, booking questions, buffer before/after, minimum notice, maximum booking horizon, slot interval, daily booking limits, cancellation/rescheduling policy, and reminder policy.

Availability rules store an IANA timezone and recurring local-time intervals for each weekday. Support date-specific overrides and time off. Convert local availability to concrete instants correctly, including daylight-saving transitions where relevant. Store booking instants in UTC and preserve the intended timezone for display and invitations.

Calculate available slots from event settings, availability rules, time off, existing active bookings, buffers, interviewer conflicts, booking limits and scheduling horizon. A slot is valid only if the entire event duration and required buffers fit. Never rely on the frontend's displayed slots as authoritative.

On booking, re-check availability on the server and use a concurrency-safe transaction, lock, or other proven mechanism to prevent double-booking. Use idempotency keys or equivalent deduplication for booking creation. Return a clear conflict response if another request claims the slot first.

Support manual scheduling by authorized interviewers and public self-service booking. Both must use the same booking service and invariants. A booking record should have a stable ID, participants, event type, UTC start/end, timezone, status, meeting provider/reference, questions/answers, timestamps, version, and cancellation/reschedule metadata.

Implement a state machine for booking statuses such as pending (only if required), confirmed, cancelled, completed, and no-show. Validate legal transitions server-side. Rescheduling must safely reserve the new slot, update the booking, invalidate old notification jobs, create new jobs, and notify attendees. Cancellation must release the slot and invalidate pending

reminders.

Generate unique, unguessable public booking slugs and meeting access identifiers. Slugs are not authorization tokens for private records. Add optional booking limits, cancellation cutoffs and anti-abuse controls.

9. Email, calendar invitations, OTP and reminders

Create reusable, responsive templates for OTP verification, welcome/account notices, booking confirmation, rescheduling, cancellation, reminders, interview follow-up, pending scorecard notification, password reset and relevant administrative alerts. Use the actual recipient and booking data; never hardcode names or appointment details.

Use a configured email provider adapter. Store provider message IDs, status, attempts, timestamps, and safe error metadata. Implement retries with exponential backoff and jitter, idempotency/deduplication, rate limiting, dead-letter/failed job handling, operational visibility, and safe manual retry controls.

Use BullMQ/Redis or an equivalent persistent job system for reminders. Reminder offsets are configurable by event type/user/organization, within safe bounds. Schedule jobs based on the booking's UTC start time; each job must reload the current booking and verify status, version, recipient preferences and intended send time before sending. Rescheduling/cancellation must invalidate stale jobs.

Use a transactional outbox or equivalent reliable event handoff so booking persistence and notification enqueueing cannot silently diverge. Document how the worker recovers after restart and how duplicate delivery is prevented. Email delivery is not guaranteed; report provider acceptance separately from confirmed delivery where the provider supports that distinction.

Create an iCalendar invitation (.ics) with correct UTC timestamps, timezone context, unique UID, duration, summary, description, and meeting link. Send update/cancellation messages on changes. If Google Calendar synchronization is implemented, use the Calendar API with explicit OAuth scopes, token security, sync state, conflict handling and disconnect support. Do not claim two-way sync if only .ics emails are implemented.

Default reminder examples may include 24 hours and 15 minutes before the interview, but load actual policies from configuration or event settings. Respect user notification preferences and avoid sending reminders for cancelled or superseded bookings.

10. Meetings, real-time communication and WebRTC

Create a meeting session tied to a valid booking. Verify booking status, participant identity, time policy, and meeting permissions before issuing room access. Use unguessable room IDs and short-lived access grants where appropriate. A URL alone must not authorize entry.

Use Socket.IO for signaling, chat, presence, typing/room events, and collaboration messages. Authenticate the Socket.IO handshake, authorize room joins, validate every event payload, enforce room membership, apply rate limits, and clean up presence on disconnect.

Use WebRTC for media streams. Configure STUN and TURN from environment/configuration; use authenticated TURN credentials and document provider requirements. Handle device permissions, denied devices, network changes, reconnects, ICE failures, participant departure and graceful cleanup. Do not claim reliable connectivity without testing relay scenarios.

Use a managed video service instead of raw WebRTC if time or reliability requires it, but isolate it behind a meeting-provider interface. Store provider meeting references and status, not fabricated meeting URLs. Keep video and audio recording disabled unless a complete consent, retention, access and deletion workflow is explicitly implemented.

11. Collaborative coding and safe execution

Integrate Monaco Editor with language selection, question statement, examples, constraints, editor settings, run/submit actions, execution status, stdout/stderr, compile errors, test results and submission history. Persist relevant code submissions under the correct user and booking with access control.

Implement collaborative editing using Socket.IO or an appropriate collaboration protocol. Use server-authorized rooms, versioning or operational conflict handling, reconnect recovery, payload limits, and clear synchronization state. Do not broadcast private submissions to unauthorized participants.

Code execution must occur in a separate isolated service such as Judge0 or a hardened sandbox. Never use eval, child_process on the main app server, or unrestricted containers to run arbitrary user code. Enforce CPU/memory/time limits, process limits, output caps, filesystem restrictions, no privileged execution, network restrictions, and cleanup. Keep provider credentials server-side.

Validate language against supported server configuration, apply per-user and per-interview quotas, prevent execution abuse, and record actual results from the execution provider. Hidden test cases and expected outputs must not be returned to the candidate before permitted. Handle provider timeout, quota exhaustion, compile error, runtime error, and unavailable service explicitly.

12. Question bank, feedback and assessments

Question records support title, statement, topic tags, difficulty, type (coding/MCQ/system design/behavioral), constraints, examples, starter code, supported languages, evaluation configuration, visibility, owner and timestamps. Hidden tests and answer keys must be accessible only to authorized server-side evaluation code.

Allow authorized users to create question sets and attach them to event types or bookings. Validate access before exposing question content. Record assessment attempts, submitted answers, code, language, execution results, scores and timestamps. Separate candidate-visible feedback from interviewer-private notes.

Scorecards should be configurable, with criteria such as problem-solving, correctness, communication and role-specific skills. Support rating scales, written evidence, strengths, concerns and recommendation. Do not silently infer hiring decisions from analytics or AI output. Make AI-generated summaries or suggestions optional, clearly labeled and reviewable by a human.

13. Analytics and reporting

Calculate dashboard metrics from real persisted data using aggregation pipelines or dedicated reporting queries. Possible metrics: bookings over time, cancellation/no-show rates, interview completion, time-to-schedule, pending scorecards, assessment scores, question usage, and interviewer workload. Clearly define each metric and its time range.

Apply role-based data scopes: candidates see only their own data and permitted feedback; interviewers see their authorized bookings and evaluations; administrators see only the data permitted by policy. Filter by time range, event type, status and relevant owner. Use pagination and indexes for large datasets.

Do not generate fake chart values in production. If there is no data, display an honest empty state with a link to create the first event or booking. Demo seed records must be clearly labeled and excluded from production data.

14. MongoDB data model

Design Mongoose schemas with field types, required constraints, enums, indexes, timestamps, ownership references and validation. At minimum define: User, AuthIdentity, Session, EmailVerification/OTPChallenge, EventType, AvailabilityRule, TimeOff, Booking, BookingQuestion/Answer, NotificationPreference, NotificationJob/OutboxEvent, MeetingSession, ChatMessage (if persisted), Question, QuestionSet, Submission, Feedback/Scorecard, AuditLog and Organization/Team if multi-tenant functionality is included.

Use unique indexes for provider identity and appropriate public slugs; indexes for booking owner/start/status, notification due time/status, user-owned lists and analytics filters. Avoid treating application-level checks as sufficient for uniqueness under concurrency. Document deletion/cascade behavior and protect historical records when appropriate.

Do not store raw passwords, OTPs, session tokens, OAuth access tokens or provider secrets in ordinary application records. Hash or encrypt sensitive tokens as appropriate and store only what is needed. Use a consistent approach for references, pagination, schema validation and migrations/data backfills.

15. API design and error contract

Use versioned or consistently namespaced REST endpoints, for example /api/auth, /api/users, /api/event-types, /api/availability, /api/bookings, /api/meetings, /api/questions, /api/submissions, /api/feedback and /api/analytics. Document every endpoint with method, authentication, permissions, input schema, success response, error codes and side effects.

Use consistent JSON responses and HTTP status codes. Validate all request bodies, query parameters and route parameters. Return 400 for invalid input, 401 for unauthenticated requests, 403 for forbidden access, 404 for missing/inaccessible resources where appropriate, 409 for booking conflicts, 422 for semantic validation errors when useful, and 429 for rate limits. Never expose internal exception details in production.

Add pagination and filtering to list endpoints. Use idempotency for operations where retries could duplicate side effects. Keep controller logic thin and business invariants in testable services. Avoid overly generic abstractions that obscure the workflow.

16. Testing requirements

Unit tests: OTP expiry/reuse/attempt limits; password and session logic; OAuth callback state handling; authorization policies; availability slot calculation; timezone/DST cases; booking state transitions; concurrent booking conflict prevention; reminder scheduling and invalidation; analytics aggregation; input validation.

Integration tests: registration and OTP verification; login/logout; Google OAuth behavior using a mocked provider boundary; public booking; reschedule/cancel; email provider adapter; queue worker retries; question access control; submission creation; feedback permissions; admin authorization.

End-to-end tests with Playwright: register/login; verify email using a test-only mail capture service; configure event type; publish booking link; book a slot; confirm email; reschedule/cancel; join a test meeting room; submit feedback; verify dashboard metrics. Test-only shortcuts must be disabled in production and protected by environment checks.

Security tests: IDOR/BOLA, role escalation, CSRF where applicable, NoSQL injection, XSS, brute-force OTP, rate limits, session revocation, unauthorized Socket.IO joins, hidden-test leakage, malicious code execution boundaries, and secret leakage in logs/responses.

Use deterministic fixtures and isolated test databases. Never rely on real production accounts, live email recipients or unbounded external API calls in automated tests. Add CI steps for linting, tests, build, dependency/security checks where available, and fail the pipeline on failures.

17. Observability, reliability and deployment

Use structured logs with request IDs, job IDs and correlation IDs. Add health/readiness endpoints that do not expose secrets. Track queue depth, failed email jobs, booking conflicts, WebRTC signaling failures, code execution failures and API latency. Provide safe admin diagnostics and clear operational runbooks.

Provide Dockerfiles and a local development setup for frontend, backend, MongoDB and Redis, using supported services or managed equivalents. Add a CI workflow, environment validation, build scripts, seed scripts and documented migration/data-update procedures.

Production deployment must use HTTPS, secure cookies, restricted CORS, managed secrets, least-privilege credentials, database backups, error monitoring and explicit allowed origins. Configure frontend and backend URLs from environment variables. Document provider setup and deployment steps. Do not include real secrets in the repository.

Add graceful shutdown for HTTP server, Socket.IO, workers and database connections. Make background jobs restart-safe and idempotent. Add retention policies and backup/restore guidance. Provide a demo mode only if it is clearly separated from real production accounts and data.

18. Suggested repository structure

Use a clean monorepo or clearly separated frontend/backend directories. Suggested structure:

```
interviewhub/\n client/\n src/\n app/\n components/\n features/\n auth/ scheduling/ bookings/ meetings/
coding/ feedback/ analytics/\n hooks/ lib/ routes/ styles/\n server/\n src/\n config/\n middleware/\n
modules/\n auth/ users/ eventTypes/ availability/ bookings/ notifications/ meetings/ questions/ submissions/
feedback/ analytics/ admin/\n jobs/ workers/ providers/ utils/\n app.js server.js\n tests/\n shared/ (only if
useful for shared schemas/constants)\n docs/\n docker-compose.yml\n .env.example\n README.md
```

Adjust the structure if needed, but keep secrets/configuration, route definitions, business services, data access, external providers, background workers and tests clearly separated. Do not create an unnecessarily complex architecture just to match a folder tree.

19. Delivery requirements and definition of done

Deliver complete source code, a detailed README, architecture diagram, environment variable reference, API documentation, database model notes, security notes, setup instructions, provider configuration instructions, test commands, deployment guide, demo seed instructions and troubleshooting section.

All key workflows must be connected end-to-end. No dead buttons, fake success messages, placeholder analytics presented as real, hardcoded user data, committed secrets, unimplemented critical API handlers, or unsafe code execution. If a provider requires credentials that are unavailable, implement the integration boundary, provide exact setup instructions, and clearly mark the feature as requiring configuration rather than pretending it works.

Include responsive design, accessibility basics, loading/empty/error states, form validation, robust error handling, structured logging, tests and production configuration. Report honestly which features are complete, require provider credentials, or remain intentionally out of scope.

Do not stop after generating a plan or UI mockups. Implement the project incrementally, run the available checks, fix errors, verify critical workflows and summarize actual verification results. Never claim tests passed unless they were run and passed.

20. Execution instructions for the coding agent

First inspect the existing repository and preserve useful working code. If starting from an empty repository, initialize the structure and scripts. Then implement in vertical slices in this order: (1) project foundation and configuration, (2) authentication/OTP/session security, (3) profiles and roles, (4) event types and availability, (5) booking and conflict safety, (6) email templates and durable reminder jobs, (7) dashboards and public booking flow, (8) meeting integration, (9) collaborative coding and isolated execution, (10) feedback/analytics/admin, (11) tests, documentation and deployment.

At each slice: define schemas and API contracts; implement backend logic; build the corresponding UI; add validation, permissions and failure states; write tests; run lint/build/tests; and fix issues before continuing. Keep a checklist of completed work backed by actual test or manual verification. Do not replace a failing integration with fake data just to make the UI look complete.

When a design or provider choice is uncertain, choose the simplest secure, maintainable option that meets the product requirements, document the trade-off, and continue. Ask a question only when a missing decision blocks safe implementation. Produce a final report listing implemented features, setup requirements, commands run, test outcomes, external credentials/services still needed, and known limitations.